Daily Commit – PRD

Overview

Daily Commit is a group-based coding practice platform that turns interview prep into a daily *group* habit. Members solve a shared daily problem in an in-browser editor with real code execution, keep a group streak alive, and run mock interviews with peers.

The chapter's monthly assessments (on the website) measure where pledges stand. Daily Commit is the perfect practice that gets the PC ready for it.

- **Capstone track:** Computer Science / ITI
- **Checkpoint:** 1 — Project Proposal

Problem Statement

Most people start interview prep and quit because solo practice has no accountability and solved patterns are forgotten within weeks.

- **The problem:** LeetCode and NeetCode are built for individuals grinding alone. Groups that want to prep together have no shared structure, and between the chapter's monthly assessments, nothing keeps people practicing.
- **Who is affected:** CS students preparing for internship interviews, especially groups prepping together (pledge classes, clubs, study groups).
- **Why it matters:** Interview success depends on consistent practice and retention. Streaks and peer accountability are proven motivators, but no coding prep tool applies them for groups.
- **How Daily Commit addresses it:** a shared daily problem, a group streak that everyone keeps alive together, live peer competition, and spaced repetition so solved problems actually stick.

Goals and Non-Goals

Goals

- Make daily coding practice a group habit through shared problems and streaks
- Run untrusted user code safely and at scale
- Support live, real-time competition and mock interviews between members
- Improve retention by resurfacing problems members struggled with

Non-goals

- Replacing the chapter's monthly assessments (Daily Commit complements them)
- Competing with LeetCode's problem library (a curated original set of 30–40 problems should be enough)

Target Users

The first user group is my pledge class, with a path to any group prepping together.

User	Need
Group member	A daily nudge to practice, peers to compete with, and review of weak topics
Group leader (e.g. a VP or club officer)	Visibility into who is practicing and which topics the group struggles with

Features and Requirements

P0 features make up the MVP for the midpoint demo; P1 features complete the final capstone; P2 is a stretch.

Feature	Requirement	Priority
Code execution	Submit Python, Java, or C++ in a browser editor; run against hidden test cases; return a verdict per test	P0
Groups	Create or join a group with an invite code	P0

Daily problem + group streak	Same problem for all members each day; group streak continues only if everyone solves it	P0
Leaderboards	Daily and weekly rankings within a group	P0
Live 1v1 VS mode	Race a member on the same problem with live opponent progress; ELO rating updates after each match	P1
Spaced repetition	Problems a member struggled with resurface at increasing intervals	P1
Mock interview mode	Pair members as interviewer and candidate in a shared real-time editor with a rubric	P1
Peer solution review	Solutions unlock after the daily cutoff; members can comment on each other's code	P1
Group weakness analytics	Dashboard of topics the group fails most often	P1
Discord bot	Posts the daily problem, announces solves, warns before streaks expire	P1
AI hints	Hints that guide toward a solution without revealing it	P2

Technical Design

The API server never runs user code: submissions go through a Redis queue to sandboxed execution workers, and verdicts stream back to the browser over WebSockets.

Stack: Next.js + TypeScript + Monaco Editor (frontend), FastAPI (backend), PostgreSQL (database), Redis (queue, cache, pub/sub), Docker (sandboxing), Discord API, Docker Compose on a cloud VPS. Submissions supported in Python, Java, C++.

Required components

- **Frontend:** problem pages, code editor, group dashboard, leaderboards, verses mode, and mock interview screens
- **Backend:** REST and WebSocket API for submissions, groups, matches, reviews, and scheduling
- **Database:** users, groups, problems, test cases, submissions, streaks, ratings, review schedules, comments

Additional technical components

#	Component	Category	How it's used
1	**Sandboxed execution service**	**Microservices / distributed systems**	**Separate service runs untrusted code in isolated containers with CPU, memory, time, and network limits; workers scale horizontally**
2	**Submission queue + scheduled jobs**	**Background jobs / task queues**	**Redis queue feeds execution workers; scheduled jobs assign the daily problem, evaluate streaks, unlock solutions, send reminders**
3	**Live verdicts, VS mode, activity feed**	**Real-time / WebSockets**	**Test-by-test verdict streaming, live opponent progress, group activity feed**
4	**SM-2 spaced repetition + ELO rating**	**Algorithm**	**Schedules each member's reviews by performance; ranks and matches VS mode players**

5	**Discord bot**	**Third-party service integration**	**Daily problem posts, solve announcements, streak warnings**

Success Metrics and Validation

Daily Commit succeeds if the sandbox holds up under attack and load, and if my pledge class actually uses it.

Area	How it's validated
Sandbox security	Adversarial submissions (infinite loops, memory exhaustion, fork bombs, network access attempts) are all contained
Performance	Load test with concurrent submissions; report queue throughput and verdict latency
Correctness	Unit tests for SM-2 scheduling, ELO updates, and streak logic; every problem has hand-written edge-case tests
Adoption	Pledge class usage during pledging: active members, problems solved, longest group streak

Milestones

The P0 MVP (code execution, groups, daily problem, streaks) is working end to end by the midpoint demo; P1 features land before the final capstone.

[embed: node/18a57d61-400a]

Checkpoint dates to be confirmed with the VP of Tech and Prof Development.

Risks and Open Questions

Scope is the biggest risk; P1 features are ordered so any can be cut without breaking the MVP.

Risk	Mitigation
Sandbox escape or abuse	Proven isolation (Docker with no network, dropped capabilities, CPU, memory, time, and process limits) instead of a custom sandbox
Scope creep over six weeks	P0 MVP first; P1 features built in priority order; AI hints only if time allows
Problem content	Original problem set of 30–40 problems; no scraping of LeetCode or NeetCode
Low usage by the pledge class	Discord bot reminders and group streaks drive daily use

Open questions

- Does the chapter's assessment tool already offer analytics? If so, drop group weakness analytics.
- Go instead of Python for the execution service?
- Checkpoint dates for CP2–CP4